

알고리즘이 갖추어야 할 다섯 가지 조건 중 '입력(input)'에 대한 설명으로 옳은 것은?

- ① 적어도 1개는 주어져야 한다.
- ② 입력이 없는 절차는 절대 알고리즘이 될 수 없다.
- ③ 출력의 개수와 같은 수만큼 주어져야 한다.
- ④ 사용자가 키보드로 직접 입력한 값이어야 한다.
- ⑤ 0개 이상이면 되므로 반드시 있어야 하는 것은 아니다.



알고리즘이 갖추어야 할 다섯 가지 조건 중 '입력(input)'에 대한 설명으로 옳은 것은?



정답

⑤ 0개 이상이면 되므로 반드시 있어야 하는 것은 아니다.



해설

입력은 알고리즘을 시작할 때 외부에서 받는 데이터이다. 입력을 받지 않고 1부터 100까지의 합을 출력하는 절차도 알고리즘이 될 수 있다. 따라서 입력은 0개 이상이며, 출력은 적어도 하나 있어야 한다.



다음 중 알고리즘의 유한성 조건을 만족하지 않는 것은? 다른 조건은 충족한다고 가정한다.

- ① 종료 조건 없이 같은 명령을 계속 반복하는 절차
- ② 1부터 100까지 더한 값을 출력하는 절차
- ③ 두 수의 최대공약수를 구해 반환하는 절차
- ④ 리스트에서 최대값을 찾아 반환하는 절차
- ⑤ 입력 없이 "Hello"를 한 번 출력하는 절차

다음 중 알고리즘의 유한성 조건을 만족하지 않는 것은? 다른 조건은 충족한다고 가정한다.



정답 ① 종료 조건 없이 같은 명령을 계속 반복하는 절차



해설

유한성은 명령을 유한 번 실행한 뒤 끝나야 한다는 조건이다. ①은 같은 명령을 끝없이 반복하므로 이 조건을 위반한다. ⑤는 Hello를 한 번 출력하고 끝나므로 입력이 없어도 유한성을 만족한다.



요리 절차에 "소금을 적당히 넣는다"라고 적혀 있다. 이 명령이 위반하는 알고리즘의 조건은?

① 입력

② 출력

③ 유효성

④ 유한성

⑤ 명백성



요리 절차에 "소금을 적당히 넣는다"라고 적혀 있다. 이 명령이 위반하는 알고리즘의 조건은?



정답

⑤

명백성



해설

명백성(clarity) 위반이다. '적당히'는 사람마다 다르게 해석되므로 모호하다. "소금 3g을 넣는다"처럼 해석의 여지가 없어야 한다.



어떤 절차에 "숫자 x 를 0으로 나눈 값을 변수 y 에 저장한다"라는 명령이 있다. 이 명령이 위반하는 알고리즘의 조건은?

① 입력

② 유효성

③ 명백성

④ 유한성

⑤ 출력



어떤 절차에 "숫자 x 를 0으로 나눈 값을 변수 y 에 저장한다"라는 명령이 있다. 이 명령이 위반하는 알고리즘의 조건은?



정답

②

유효성



해설

유효성(effectiveness) 위반이다. 각 명령은 실제로 수행 가능한 기본 연산이어야 하는데, 0으로 나누기는 수행할 수 없다. 명령이 모호한 것이 아니라(명백성은 만족) 아예 실행이 불가능하다는 점에서 명백성과 구별된다.

리스트 $S = [9, 3, 2, 8, 10, 1]$ 에서 다음 Python 코드를 실행한다. `linear_search(S, 8)`의 반환값은? 리스트의 인덱스는 0부터 시작한다.

code.py

```
def linear_search(S, target):  
    for i in range(len(S)):  
        if S[i] == target:  
            return i  
    return -1
```

① 3

② 4

③ 8

④ -1

⑤ 0

리스트 $S = [9, 3, 2, 8, 10, 1]$ 에서 다음 Python 코드를 실행한다. $\text{linear_search}(S, 8)$ 의 반환값은? 리스트의 인덱스는 0부터 시작한다.

code.py

```
def linear_search(S, target):  
    for i in range(len(S)):  
        if S[i] == target:  
            return i  
    return -1
```



정답

① 3



해설

$i=0, 1, 2$ 일 때 읽는 값은 9, 3, 2이므로 8과 다르다. $i=3$ 에서 $S[3]=8$ 이 되어 $\text{return } i$ 를 실행한다. 따라서 반환값은 3이다. 찾은 값 8이나 네 번째라는 순서 4를 반환하는 것은 아니다.

순서도(flowchart)에서 조건 판단을 나타내는 도형은?

① 타원

② 직사각형

③ 원

④ 평행사변형

⑤ 마름모

순서도(flowchart)에서 조건 판단을 나타내는 도형은?



정답 ⑤ 마름모



해설

타원은 시작/끝, 직사각형은 처리, 마름모는 조건 판단, 평행사변형은 입출력이다. 마름모에서는 조건의 참·거짓에 따라 다음 경로가 갈라진다.

한국어처럼 일상에서 쓰는 말(자연어)로 알고리즘의 절차를 설명할 때 생길 수 있는 대표적인 단점은?

- ① 작성하는 데 시간이 오래 걸린다는 점이 가장 큰 문제다.
- ② 비전공자는 내용을 전혀 이해할 수 없다는 점이 문제다.
- ③ 표현이 모호하면 읽는 사람마다 다르게 해석할 수 있다.
- ④ 반복이나 조건 같은 논리 구조를 절대 표현할 수 없다는 점이 문제다.
- ⑤ 종이에 옮겨 적을 수 없다는 점이 가장 큰 문제다.

한국어처럼 일상에서 쓰는 말(자연어)로 알고리즘의 절차를 설명할 때 생길 수 있는 대표적인 단점은?



정답 ③ 표현이 모호하면 읽는 사람마다 다르게 해석할 수 있다.



해설

예를 들어 "값을 적당히 크게 만든다"라고만 쓰면 얼마만큼 바꿀지 정해져 있지 않다. 자연어로 설명할 때 이런 모호함이 생길 수 있다. 반면 "현재 값에 3을 더한다"처럼 구체적으로 쓰면 의미가 분명해진다.

다음 Python 코드를 `gcd_brute_force(1000000, 999999)`로 실행한다. `while` 안의 `if`문은 총 몇 번 실행되는가? %는 나머지 연산이다.

code.py

```
def gcd_brute_force(a, b):  
    t = min(a, b)  
    while t > 0:  
        if a % t == 0 and b % t == 0:  
            return t  
        t -= 1  
    return 1
```

① 2회

② 20회

③ 1,000회

④ 999,999회

⑤ 10억 회

다음 Python 코드를 `gcd_brute_force(1000000, 999999)`로 실행한다. `while` 안의 `if`문은 총 몇 번 실행되는가? %는 나머지 연산이다.

```
code.py

def gcd_brute_force(a, b):
    t = min(a, b)
    while t > 0:
        if a % t == 0 and b % t == 0:
            return t
        t -= 1
    return 1
```



정답

④

999,999회



해설

t는 999,999에서 시작해 1씩 줄어든다. 두 수의 공약수는 1뿐이므로 t=1에서 처음 return을 실행한다. 따라서 if문은 t=999,999부터 t=1까지 총 999,999번 실행된다.

$\gcd(a, b)$ 는 두 수 a, b 의 최대공약수이다. $a \bmod b$ 는 a 를 b 로 나눈 나머지이다. a 와 b 가 양의 정수일 때, 유클리드 알고리즘에서 항상 성립하는 관계식은?

① $\gcd(a, b) = \gcd(a \times b, b)$

② $\gcd(a, b) = \gcd(b, a \div b)$

③ $\gcd(a, b) = \gcd(a \bmod b, a + b)$

④ $\gcd(a, b) = \gcd(a + b, a - b)$

⑤ $\gcd(a, b) = \gcd(b, a \bmod b)$

$\gcd(a, b)$ 는 두 수 a, b 의 최대공약수이다. $a \bmod b$ 는 a 를 b 로 나눈 나머지이다.
 a 와 b 가 양의 정수일 때, 유클리드 알고리즘에서 항상 성립하는 관계식은?



정답 ⑤ $\gcd(a, b) = \gcd(b, a \bmod b)$



해설

a 를 b 로 나눈 몫을 q , 나머지를 r 이라 하면 $a = q \times b + r$ 이다. a 와 b 를 모두 나누는 수는 $r = a - q \times b$ 도 나눈다. 반대로 b 와 r 을 모두 나누는 수는 $a = q \times b + r$ 도 나눈다. 공약수가 같으므로 최대공약수도 같아 $\gcd(a, b) = \gcd(b, r)$ 이다.

$a=48$, $b=18$ 에서 시작한다. b 가 0이 아니면 두 값을 동시에 $a \leftarrow b$, $b \leftarrow a \bmod b$ 로 바꾼다. $b=0$ 이면 a 를 반환한다.

$\bmod$ 는 나머지 연산이다. 나머지는 총 몇 번 계산하며, 마지막에 무엇을 반환하는가?

① 2회, 6

② 4회, 6

③ 3회, 12

④ 3회, 6

⑤ 2회, 12

$a=48$, $b=18$ 에서 시작한다. b 가 0이 아니면 두 값을 동시에 $a \leftarrow b$, $b \leftarrow a \bmod b$ 로 바꾼다. $b=0$ 이면 a 를 반환한다. $\bmod$ 는 나머지 연산이다. 나머지는 총 몇 번 계산하며, 마지막에 무엇을 반환하는가?



정답 ④ 3회, 6



해설

값의 쌍 (a,b) 는 $(48,18) \rightarrow (18,12) \rightarrow (12,6) \rightarrow (6,0)$ 으로 바뀐다. 나머지 계산은 $48 \bmod 18$, $18 \bmod 12$, $12 \bmod 6$ 의 3번이다. $b=0$ 이 되면 그때의 a 인 6을 반환한다.

두 양의 정수 a, b 의 최대공약수를 다음 두 방법으로 구한다.

완전 탐색: 두 수 중 작은 값부터 1씩 줄이며 첫 공약수를 찾는다.

유클리드: $b \neq 0$ 인 동안 (a, b) 를 $(b, a \bmod b)$ 로 동시에 바꾸고, $b = 0$ 이면 a 를 반환한다.

$\bmod$ 는 나머지이다. 옳지 않은 설명은?

- ① 두 방법을 모두 10배 빠른 컴퓨터에서 실행하면 연산 횟수 차이가 사라진다.
- ② 두 방법 모두 반복문을 사용해 구현할 수 있다.
- ③ 작은 쪽 입력값이 같다면 완전 탐색은 최대공약수가 작을수록 많이 반복한다.
- ④ 유클리드는 문제를 더 작은 문제로 계속 바꾸어 나간다.
- ⑤ 두 방법 모두 항상 정확한 최대공약수를 낸다.

두 양의 정수 a, b 의 최대공약수를 다음 두 방법으로 구한다.

완전 탐색: 두 수 중 작은 값부터 1씩 줄이며 첫 공약수를 찾는다.

유클리드: $b \neq 0$ 인 동안 (a, b) 를 $(b, a \bmod b)$ 로 동시에 바꾸고, $b = 0$ 이면 a 를 반환한다. $\bmod$ 는 나머지이다. 옳지 않은 설명은?



정답 ① 두 방법을 모두 10배 빠른 컴퓨터에서 실행하면 연산 횟수 차이가 사라진다.



해설

컴퓨터가 빨라지면 명령 하나에 걸리는 시간이 줄어든다. 그러나 같은 입력에 같은 코드를 실행할 때 반복 횟수는 달라지지 않는다. 예를 들어 $(1,000,000, 999,999)$ 에서 두 방법의 반복 횟수는 각각 999,999회와 2회이다.

가운데 값을 확인한 뒤 왼쪽이나 오른쪽 절반으로 범위를 줄이는 이진 탐색을 하려고 한다. 데이터가 만족해야 하는 조건은?

- ① 리스트에 저장된 값이 모두 양의 정수여야 한다.
- ② 데이터의 개수가 반드시 2의 거듭제곱이어야 한다.
- ③ 데이터에 같은 값이 두 번 이상 나타나지 않아야 한다.
- ④ 데이터가 오름차순이나 내림차순으로 반드시 정렬되어 있어야 한다.
- ⑤ 찾으려는 값이 반드시 리스트 안에 있어야 한다.

가운데 값을 확인한 뒤 왼쪽이나 오른쪽 절반으로 범위를 줄이는 이진 탐색을 하려고 한다. 데이터가 만족해야 하는 조건은?



정답

④ 데이터가 오름차순이나 내림차순으로 반드시 정렬되어 있어야 한다.



해설

가운데 값과 찾는 값을 비교해 어느 쪽을 버릴지 결정하려면 데이터가 정렬되어 있어야 한다. 오름차순에서는 가운데 값보다 큰 값이 오른쪽에 있다. 내림차순에서는 방향을 반대로 판단한다. 찾는 값이 없더라도 이진 탐색으로 없음을 확인할 수 있다.

정렬된 숫자 1,000,000개에서 이진 탐색으로 값을 찾는다. 가운데 원소를 확인하는 일을 1회로 센다. 찾지 못하면 그 원소를 제외한 왼쪽이나 오른쪽 구간을 계속 탐색한다. 값을 찾거나 후보가 없어질 때까지 필요한 최대 확인 횟수는?

① 10회

② 20회

③ 30회

④ 1,000회

⑤ 1,000,000회

정렬된 숫자 1,000,000개에서 이진 탐색으로 값을 찾는다. 가운데 원소를 확인하는 일을 1회로 센다. 찾지 못하면 그 원소를 제외한 왼쪽이나 오른쪽 구간을 계속 탐색한다.
값을 찾거나 후보가 없어질 때까지 필요한 최대 확인 횟수는?



정답 ② 20회



해설

이진 탐색의 최대 확인 횟수는 $\lfloor \log_2 n \rfloor + 1$ 이다. $\lfloor x \rfloor$ 는 x 이하의 가장 큰 정수이다. $\log_2(1,000,000) \approx 19.9$ 이므로 $19 + 1 = 20$ 회이다. 구간을 줄인 뒤 마지막 원소도 확인해야 한다.

정렬된 원소가 1,000,000개다. 순차 탐색은 최대 1,000,000회, 이진 탐색은 약 20회 비교한다. 순차 탐색의 비교 횟수는 이진 탐색의 약 몇 배인가?

① 5배

② 20배

③ 1,000배

④ 50,000배

⑤ 1,000,000배

정렬된 원소가 1,000,000개다. 순차 탐색은 최대 1,000,000회, 이진 탐색은 약 20회 비교한다. 순차 탐색의 비교 횟수는 이진 탐색의 약 몇 배인가?



정답

④

50,000배



해설

$1,000,000 \div 20 = 50,000$ 이다. 같은 입력에서도 탐색 방법에 따라 비교 횟수가 크게 달라진다.

다음 Python 코드는 오름차순 숫자 리스트 S에서 x를 찾는다. //는 나눗셈의 정수 몫이다. low = mid + 1을 low = mid로 바꾸고 나머지는 그대로 두면, 어떤 문제가 생길 수 있는가?

```
code.py
def binary_search(S, x):
    low = 0
    high = len(S) - 1
    while low <= high:
        mid = (low + high) // 2
        if S[mid] == x:
            return mid
        elif S[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

- ① low = mid가 Python 문법에 맞지 않아 실행할 수 없다.
- ② 같은 범위를 계속 확인하여 반복문이 끝나지 않을 수 있다.
- ③ 값이 리스트에 있어도 -1을 반환할 수 있다.
- ④ 리스트 원소들의 순서가 뒤바뀔 수 있다.
- ⑤ 리스트에 새 원소가 추가될 수 있다.

다음 Python 코드는 오름차순 숫자 리스트 S에서 x를 찾는다. //는 나눗셈의 정수 몫이다. low = mid + 1을 low = mid로 바꾸고 나머지는 그대로 두면, 어떤 문제가 생길 수 있는가?

code.py

```
def binary_search(S, x):
    low = 0
    high = len(S) - 1
    while low <= high:
        mid = (low + high) // 2
        if S[mid] == x:
            return mid
        elif S[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```



정답
있다.

② 같은 범위를 계속 확인하여 반복문이 끝나지 않을 수



해설

예를 들어 S=[1], x=2이면 low=high=mid=0이다. S[0]<2이므로 low=mid를 실행해도 low는 0이다. 다음 반복에서도 같은 값을 확인하므로 끝나지 않는다. 원래 코드의 low=mid+1은 이미 확인한 원소를 탐색 범위에서 제외한다.

입력 크기를 n 이라 하자. "컴퓨터가 빨라졌으니 어떤 알고리즘을 쓰든 상관없다"는 주장에 대한 반박으로 적절하지 않은 것은?

- ① 다루어야 할 데이터의 크기도 함께 커졌기 때문이다.
- ② 컴퓨터 속도는 연산 한 번의 시간을 줄이지만, 알고리즘을 바꾸면 연산 횟수의 증가 방식도 바뀔 수 있다.
- ③ 하드웨어를 10배 빠르게 하면 $O(n^2)$ 이 $O(n)$ 으로 바뀐다.
- ④ 연산 횟수가 n 에 비례하는 방법과 n^2 에 비례하는 방법의 차이는 n 이 커질수록 벌어진다.
- ⑤ 큰 데이터를 처리할수록 연산 횟수가 적게 늘어나는 알고리즘의 이점이 커질 수 있다.

입력 크기를 n 이라 하자. "컴퓨터가 빨라졌으니 어떤 알고리즘을 쓰든 상관없다"는 주장에 대한 반박으로 적절하지 않은 것은?



정답

③ 하드웨어를 10배 빠르게 하면 $O(n^2)$ 이 $O(n)$ 으로 바뀐다.



해설

③의 주장은 틀리다. 같은 코드를 10배 빠른 컴퓨터에서 실행해도 연산 횟수는 그대로이다. n^2 에 비례하던 연산 횟수가 n 에 비례하도록 바뀌지는 않는다. 실행 시간이 일정한 비율로 줄어든 뿐이다.

알고리즘을 비교할 때, 시간 복잡도 분석에서 실행 시간(초) 대신 기본 연산의 횟수를 세는 주된 이유는?

- ① 컴퓨터와 언어에 따라 달라져 알고리즘 고유의 성질이 아니기 때문이다.
- ② 실행 시간을 정확히 측정할 도구가 존재하지 않기 때문이다.
- ③ 초 단위 값은 자릿수가 너무 커서 서로 비교하기 어렵기 때문이다.
- ④ 같은 알고리즘의 실행 시간은 환경과 무관하게 항상 일정하기 때문이다.
- ⑤ 실행 시간보다 메모리 사용량이 훨씬 더 중요한 지표이기 때문이다.

알고리즘을 비교할 때, 시간 복잡도 분석에서 실행 시간(초) 대신 기본 연산의 횟수를 세는 주된 이유는?

**정답**

① 컴퓨터와 언어에 따라 달라져 알고리즘 고유의 성질이 아니기 때문이다.

**해설**

같은 입력과 같은 코드를 실행해도 컴퓨터나 언어에 따라 걸리는 시간은 달라질 수 있다. 반면 어떤 연산을 한 번으로 셀지 정해 두면, 그 연산의 실행 횟수로 알고리즘이 해야 할 일의 양을 비교할 수 있다.

시간 복잡도를 분석할 때, 실행 횟수를 셀 대표 명령을 기본 연산이라고 한다. 기본 연산의 선택에 대한 설명으로 옳지 않은 것은?

- ① 전체 작업량이 늘어나는 정도를 잘 나타내는 명령을 고르는 것이 좋다.
- ② 보통 가장 안쪽 반복문에서 자주 실행되는 명령을 고른다.
- ③ 유일하게 정해지므로 다르게 고르면 틀린 답이 된다.
- ④ 정렬 분석에서는 목적에 따라 비교 횟수나 교환 횟수를 셀 수 있다.
- ⑤ 탐색 알고리즘에서는 보통 비교 연산을 기본 연산으로 삼는다.

시간 복잡도를 분석할 때, 실행 횟수를 셀 대표 명령을 기본 연산이라고 한다. 기본 연산의 선택에 대한 설명으로 옳지 않은 것은?



정답 ③ 유일하게 정해지므로 다르게 고르면 틀린 답이 된다.



해설

기본 연산은 분석 목적에 따라 선택한다. 비교 횟수와 교환 횟수는 서로 다른 지표이며, 전체 시간의 증가를 추정하려면 선택한 연산이 전체 비용을 대표하는지 확인해야 한다. 따라서 선택이 유일하다는 ③이 틀리다.

A는 숫자 n 개가 든 리스트이며 $n \geq 1$ 이다. 다음 Python 코드를 한 번 실행한다. $A[i] > \text{largest}$ 를 검사하는 횟수만 셀 때, 총 몇 번 검사하는가?

```
code.py
def get_largest(A):
    largest = A[0]
    for i in range(1, len(A)):
        if A[i] > largest:
            largest = A[i]
    return largest
```

① n ② $n / 2$ ③ $n + 1$ ④ $n - 1$ ⑤ $n(n-1)/2$

A는 숫자 n 개가 든 리스트이며 $n \geq 1$ 이다. 다음 Python 코드를 한 번 실행한다. $A[i] > \text{largest}$ 를 검사하는 횟수만 셀 때, 총 몇 번 검사하는가?

```
code.py

def get_largest(A):
    largest = A[0]
    for i in range(1, len(A)):
        if A[i] > largest:
            largest = A[i]
    return largest
```



정답

④

$n - 1$



해설

첫 원소 $A[0]$ 은 largest 에 넣고 시작한다. 이후 인덱스 1부터 $n-1$ 까지 각 원소를 한 번씩 비교하므로 총 $n-1$ 회이다. 숫자의 크기나 순서가 달라도 이 반복 횟수는 같다.

리스트에는 원소가 n 개 있으며 $n \geq 1$ 이다. 찾는 값 $target$ 은 정확히 한 번 들어 있다. 어느 위치에 있는지도 똑같이 가능하다(각 위치의 확률은 $1/n$). 첫 원소부터 $target$ 과 한 번씩 비교하고 찾으면 즉시 멈출 때, 평균 비교 횟수는?

① $n / 2$

② $(n + 1) / 2$

③ $(n - 1) / 2$

④ n

⑤ $\log_2 n$

리스트에는 원소가 n 개 있으며 $n \geq 1$ 이다. 찾는 값 $target$ 은 정확히 한 번 들어 있다. 어느 위치에 있는지도 똑같이 가능하다(각 위치의 확률은 $1/n$).

첫 원소부터 $target$ 과 한 번씩 비교하고 찾으면 즉시 멈출 때, 평균 비교 횟수는?



정답 ② $(n + 1) / 2$



해설

$target$ 이 첫 번째에 있으면 1회, 두 번째면 2회, 마지막이면 n 회 비교한다. 각 위치의 가능성이 같으므로 평균은 $(1+2+\dots+n)/n = (n+1)/2$ 회이다. $n/2$ 는 근삿값일 뿐 정확한 평균은 아니다.

A는 숫자 n 개가 든 리스트이다($n=\text{len}(A)$). 다음 Python 코드에서 `total + x`의 덧셈을 실행한 총횟수를 $T(n)$ 이라 하자. $T(n)$ 은?

```
code.py
def add_array_members(A):
    total = 0
    for x in A:
        total = total + x
    return total
```

① $T(n) = 1$

② $T(n) = \log n$

③ $T(n) = n$

④ $T(n) = n - 1$

⑤ $T(n) = n^2$

A는 숫자 n 개가 든 리스트이다($n=\text{len}(A)$). 다음 Python 코드에서 $\text{total} + x$ 의 덧셈을 실행한 총횟수를 $T(n)$ 이라 하자. $T(n)$ 은?

code.py

```
def add_array_members(A):
    total = 0
    for x in A:
        total = total + x
    return total
```



정답

③ $T(n) = n$



해설

리스트의 원소를 하나 읽을 때마다 $\text{total}+x$ 를 한 번 계산한다. 원소가 n 개이므로 덧셈은 정확히 n 번이고 $T(n)=n$ 이다. 처음 $\text{total}=0$ 으로 정하는 대입은 덧셈 횟수에 포함하지 않는다.

입력 크기 n 에 대한 기본 연산 횟수가 $T(n)=3n^2+5n+100$ 일 때, 최고차항만 남기고 계수를 생략한 빅오 표기는?

① $O(n^2)$

② $O(n)$

③ $O(1)$

④ $O(n^3)$

⑤ $O(2^n)$

입력 크기 n 에 대한 기본 연산 횟수가 $T(n)=3n^2+5n+100$ 일 때, 최고차항만 남기고 계수를 생략한 빅오 표기는?



정답 ① $O(n^2)$



해설

가장 높은 차수의 항은 $3n^2$ 이다. 여기에서 계수 3을 생략하면 $O(n^2)$ 이 된다. $O(n^3)$ 이나 $O(2^n)$ 도 넓은 의미의 상한이지만, 이 문제는 최고차항에 맞춘 표기를 요구하므로 ①이다.

$2n+1$ 이 $O(n)$ 임을 보이기 위해 두 양의 상수 c, n_0 를 고른다.
 n 이 n_0 이상인 모든 양의 정수일 때 $2n+1 \leq c \times n$ 이 성립해야 한다.
다음 중 이 조건을 만족하지 않는 (c, n_0) 는?

① $c = 3, n_0 = 1$

② $c = 2, n_0 = 1$

③ $c = 5, n_0 = 1$

④ $c = 10, n_0 = 5$

⑤ $c = 4, n_0 = 2$

$2n+1$ 이 $O(n)$ 임을 보이기 위해 두 양의 상수 c, n_0 를 고른다.
 n 이 n_0 이상인 모든 양의 정수일 때 $2n+1 \leq c \times n$ 이 성립해야 한다.
다음 중 이 조건을 만족하지 않는 (c, n_0) 는?



정답 ② $c = 2, n_0 = 1$



해설

$c=2$ 이면 $2n+1 \leq 2n$ 이어야 하므로 어떤 양의 n 에서도 성립하지 않는다. 나머지 네 쌍은 각 n_0 이상에서 조건을 만족한다. 가능한 증명용 상수 쌍은 여러 개지만, 이 문항의 부적합한 쌍은 ②뿐이다.

입력 크기 n 이 충분히 커질 때 다음 함수들의 증가 속도를 비교한다. 느리게 증가하는 것부터 빠르게 증가하는 순서로 옳게 나열한 것은?

$\log_2 n$ 은 밑이 2인 로그이다. <는 여기서 증가 속도의 순서를 뜻한다.

① $1 < n < \log_2 n < n \log_2 n < n^2$

② $1 < n < n \log_2 n < \log_2 n < n^2$

③ $\log_2 n < 1 < n < n^2 < n \log_2 n$

④ $1 < \log_2 n < n \log_2 n < n < n^2$

⑤ $1 < \log_2 n < n < n \log_2 n < n^2$

입력 크기 n 이 충분히 커질 때 다음 함수들의 증가 속도를 비교한다. 느리게 증가하는 것부터 빠르게 증가하는 순서로 옳게 나열한 것은?

$\log_2 n$ 은 밑이 2인 로그이다. $<$ 는 여기서 증가 속도의 순서를 뜻한다.



정답 ⑤ $1 < \log_2 n < n < n \log_2 n < n^2$



해설

1은 n 이 커져도 변하지 않는다. $\log_2 n$ 은 n 보다 느리게 증가한다. $n \log_2 n$ 은 n 보다 빠르지만 n^2 보다 느리게 증가한다. 따라서 1, $\log_2 n$, n , $n \log_2 n$, n^2 의 순서이다.

arr는 숫자 n개가 든 리스트이며 $n \geq 1$ 이다. 다음 Python 코드에서 $\text{arr}[j] > \text{arr}[j+1]$ 을 검사하는 횟수만 센다. 정확한 검사 횟수와 이를 최고차항으로 정리한 빅오 표기를 바르게 짝지은 것은?

```
code.py
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

- ① $n, O(n)$
- ② $n^2, O(n^2)$
- ③ $n(n-1)/2, O(n^2)$
- ④ $n(n+1)/2, O(n)$
- ⑤ $\log_2 n$ 회, $O(\log n)$

arr는 숫자 n개가 든 리스트이며 $n \geq 1$ 이다. 다음 Python 코드에서 $\text{arr}[j] > \text{arr}[j+1]$ 을 검사하는 횟수만 센다. 정확한 검사 횟수와 이를 최고차항으로 정리한 빅오 표기를 바르게 짝지은 것은?

code.py

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```



정답

③ $n(n-1)/2, O(n^2)$



해설

$i=0$ 일 때 $n-1$ 회, $i=1$ 일 때 $n-2$ 회, 마지막에는 1회 검사한다. 합은 $(n-1)+(n-2)+\dots+1=n(n-1)/2$ 회이다. 이를 전개하면 $(n^2-n)/2$ 이므로 빅오 표기는 $O(n^2)$ 이다.

빅오와 최악의 경우에 대한 설명으로 옳은 것은?

- ① 빅오는 언제나 최악의 경우를 가리킨다.
- ② Ω 는 최선의 경우와 같은 뜻으로 쓸 수 있다.
- ③ 최악의 경우는 Θ 로 표기하는 것이 원칙이다.
- ④ 평균의 경우에도 빅오 표기를 쓴다.
- ⑤ Θ 가 성립하면 O 는 성립하지 않는다.

빅오와 최악의 경우에 대한 설명으로 옳은 것은?



정답 ④ 평균의 경우에도 빅오 표기를 쓴다.



해설

최선·평균·최악은 어떤 입력들의 연산 횟수를 분석할지 정한다. 빅오는 그렇게 얻은 함수가 얼마나 빠르게 증가하는지 나타내는 상한이다. 예를 들어 순차 탐색의 평균 비교 횟수 $(n+1)/2$ 도 $O(n)$ 으로 쓸 수 있다. 따라서 평균에도 빅오를 쓰는 ④가 맞다.



변수 A에는 "물", B에는 "주스"라는 값이 있다. $A=B$ 를 실행하면 B의 값을 A에 복사하고, A에 있던 값은 덮어쓴다.
두 값을 서로 바꾸려 한다. $A=B$, $B=A$ 를 차례로 실행하면 왜 실패하며, 임시 변수 temp는 어떤 값을 보관해야 하는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



변수 A에는 "물", B에는 "주스"라는 값이 있다. $A=B$ 를 실행하면 B의 값을 A에 복사하고, A에 있던 값은 덮어쓴다. 두 값을 서로 바꾸려 한다. $A=B$, $B=A$ 를 차례로 실행하면 왜 실패하며, 임시 변수 temp는 어떤 값을 보관해야 하는가?



정답 $A=B$ 를 하면 원래 A의 "물"을 잃는다. 이를 먼저 temp에 보관해야 한다.



해설

$A=B$ 뒤에는 A와 B가 모두 "주스"이다. 이 상태에서 $B=A$ 를 해도 물이 복구되지 않는다. $temp=A$, $A=B$, $B=temp$ 순서로 실행하면 원래 A의 값을 보존하면서 두 값을 바꿀 수 있다.



알고리즘이 갖추어야 할 다섯 가지 조건을 모두 쓰시오.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



알고리즘이 갖추어야 할 다섯 가지 조건을 모두 쓰시오.



정답

입력, 출력, 명백성, 유한성, 유효성



해설

입력은 0개 이상, 출력은 1개 이상이라는 점을 짚지어 기억하면 좋다.



같은 값이 들어 있는 리스트도 작은 값부터 정렬할 수 있어야 한다.
정렬 조건을 "앞의 값보다 반드시 커야 한다"가 아니라 "앞의 값보다 작아지지 않아야 한다(비감소)"라고 정하는 이유를 쓰시오.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



같은 값이 들어 있는 리스트도 작은 값부터 정렬할 수 있어야 한다. 정렬 조건을 "앞의 값보다 반드시 커야 한다"가 아니라 "앞의 값보다 작아지지 않아야 한다(비감소)"라고 정하는 이유를 쓰시오.



정답

같은 값이 이웃할 수 있으므로, 앞뒤 값이 같아도 허용해야 하기 때문이다.



해설

예를 들어 [1, 3, 3, 5]는 작은 값부터 정렬되어 있다. 가운데의 3 다음에 3이 오므로 매번 커지지는 않지만 작아지지도 않는다. 비감소 조건은 이처럼 같은 값이 이어지는 경우를 허용한다.



길이가 n 인 리스트의 인덱스는 0부터 $n-1$ 까지이다. 순차 탐색은 값을 찾으면 그 인덱스를 반환한다.
찾는 값이 없을 때는 왜 0 대신 -1 을 반환하도록 정하는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



길이가 n 인 리스트의 인덱스는 0부터 $n-1$ 까지이다. 순차 탐색은 값을 찾으면 그 인덱스를 반환한다. 찾는 값이 없을 때는 왜 0 대신 -1 을 반환하도록 정하는가?



정답 0은 첫 원소의 인덱스와 겹치지만, -1 은 반환할 정상 인덱스와 겹치지 않아 "없음"을 구별할 수 있기 때문이다.



순서도에서 ① 시작과 끝, ② 처리, ③ 입출력을 나타내는 도형을 각각 쓰시오.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

순서도에서 ① 시작과 끝, ② 처리, ③ 입출력을 나타내는 도형을 각각 쓰시오.



정답 ① 타원 ② 직사각형 ③ 평행사변형



해설

조건 판단은 마름모이며, 조건의 참·거짓에 따라 다음 경로가 갈라진다.

유사코드는 특정 프로그래밍 언어의 문법 대신 핵심 절차를 적는 방법이다. 값을 변수에 저장하는 대입을 $x \leftarrow 3$ 처럼 나타내기도 한다. 수학의 $x=3$ 이라는 표현과 비교할 때, 대입에 $\leftarrow$ 를 쓰면 어떤 혼동을 줄일 수 있는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

유사코드는 특정 프로그래밍 언어의 문법 대신 핵심 절차를 적는 방법이다. 값을 변수에 저장하는 대입을 $x \leftarrow 3$ 처럼 나타내기도 한다.

수학의 $x=3$ 이라는 표현과 비교할 때, 대입에 $\leftarrow$ 를 쓰면 어떤 혼동을 줄일 수 있는가?



정답 두 값이 같다는 뜻의 $=$ 와, 오른쪽 값을 왼쪽 변수에 저장하라는 대입을 구별할 수 있다.

두 양의 정수의 최대공약수를 구한다. 두 수 중 작은 값부터 1씩 줄이며 두 수를 모두 나누어떨어지게 하는 수를 찾을 때, 가장 먼저 발견한 수가 최대공약수인 이유를 한 문장으로 쓰시오.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

두 양의 정수의 최대공약수를 구한다. 두 수 중 작은 값부터 1씩 줄이며 두 수를 모두 나누어떨어지게 하는 수를 찾을 때, 가장 먼저 발견한 수가 최대공약수인 이유를 한 문장으로 쓰시오.



정답 큰 값부터 내려오며 검사하므로 가장 먼저 발견되는 공약수가 가장 큰 공약수이기 때문이다.



해설

반대로 1부터 올라가며 검사하면 끝까지 다 본 뒤에야 최대값을 알 수 있다.

다음 Python 함수의 입력 a, b 는 양의 정수이다. %는 나머지 연산이다. while문이 더 이상 반복하지 않는 조건과, 그때 return이 반환하는 변수를 각각 쓰시오.

```
code.py

def gcd(a, b):
    while b != 0:
        temp = a
        a = b
        b = temp % b
    return a
```



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

다음 Python 함수의 입력 a, b 는 양의 정수이다. %는 나머지 연산이다. while문이 더 이상 반복하지 않는 조건과, 그때 return이 반환하는 변수를 각각 쓰시오.

```
code.py

def gcd(a, b):
    while b != 0:
        temp = a
        a = b
        b = temp % b
    return a
```



정답 b 가 0이 될 때 멈추며, 그때의 a 가 반환된다.



해설

while문의 조건은 $b \neq 0$ 이다. 따라서 $b=0$ 이 되면 반복을 멈추고 return a 를 실행한다. 앞의 반복에서 최대공약수는 바뀌지 않으므로 마지막 a 가 처음 두 입력의 최대공약수이다.

다음 Python 코드에 $S=[1, 3, 5, 7, 9, 11, 13, 15]$, $x=13$ 을 넣는다. 인덱스는 0부터 시작하고, `//`는 나눗셈의 정수 몫이다. ① 처음 계산한 `mid`의 값 ② 함수의 최종 반환값을 각각 쓰시오.

```
code.py

def binary_search(S, x):
    low = 0
    high = len(S) - 1
    while low <= high:
        mid = (low + high) // 2
        if S[mid] == x:
            return mid
        elif S[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



다음 Python 코드에 $S=[1, 3, 5, 7, 9, 11, 13, 15]$, $x=13$ 을 넣는다. 인덱스는 0부터 시작하고, $//$ 는 나눗셈의 정수 몫이다. ① 처음 계산한 mid의 값 ② 함수의 최종 반환값을 각각 쓰시오.

code.py

```
def binary_search(S, x):  
    low = 0  
    high = len(S) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if S[mid] == x:  
            return mid  
        elif S[mid] < x:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

**정답** 처음 mid = 3, 반환값 6**해설**

처음에는 $low=0$, $high=7$ 이므로 $mid=3$ 이다. $S[3]=7 < 13$ 이어서 $low=4$ 가 된다. 다음 $mid=5$ 에서도 $S[5]=11 < 13$ 이다. $low=6$ 으로 바꾼 다음 $mid=6$ 에서 $S[6]=13$ 을 찾으므로 인덱스 6을 반환한다.

정렬된 숫자 n 개에서 이진 탐색을 한다($n \geq 1$). 가운데 원소를 한 번 확인하는 일을 1회로 센다. 찾지 못하면 그 원소를 제외한 한쪽 구간만 남긴다.

원소 수가 n 개에서 $2n$ 개로 늘면, 값을 찾거나 후보가 없어질 때까지 필요한 최대 확인 횟수는 몇 회 늘어나는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



정렬된 숫자 n 개에서 이진 탐색을 한다($n \geq 1$). 가운데 원소를 한 번 확인하는 일을 1회로 센다. 찾지 못하면 그 원소를 제외한 한쪽 구간만 남긴다.

원소 수가 n 개에서 $2n$ 개로 늘면, 값을 찾거나 후보가 없어질 때까지 필요한 최대 확인 횟수는 몇 회 늘어나는가?



정답 1회



해설

원소가 n 개일 때 최대 확인 횟수는 $\lfloor \log_2 n \rfloor + 1$ 이다. 원소 수를 두 배로 하면 $\log_2(2n) = \log_2 n + 1$ 이므로 이 횟수도 정확히 1 늘어난다. 예를 들어 8개는 최대 4회, 16개는 최대 5회이다.

이 수업에서는 시간 복잡도를 초 단위 실행 시간 대신 연산 횟수로 분석한다. 입력 크기 n 이 커질 때, 어떤 횟수가 얼마나 늘어나는지를 조사하는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

이 수업에서는 시간 복잡도를 초 단위 실행 시간 대신 연산 횟수로 분석한다. 입력 크기 n 이 커질 때, 어떤 횟수가 얼마나 늘어나는지를 조사하는가?



정답 비교나 덧셈 등, 분석 대상으로 정한 기본 연산의 실행 횟수



해설

입력이 커질 때 비교·대입 등 기본 연산 횟수가 어떻게 늘어나는지 평가한다.

입력 크기가 $n \geq 1$ 일 때, 어떤 알고리즘은 비교를 n^2 번, 교환을 n 번 한다. 비교와 교환 한 번의 시간은 각각 일정하며, 나머지 작업 시간은 무시한다.

교환 횟수 n 만 보고 전체 시간 복잡도를 $O(n)$ 이라고 해도 되는가? 이유와 올바른 빅오 표기를 쓰시오.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

입력 크기가 $n \geq 1$ 일 때, 어떤 알고리즘은 비교를 n^2 번, 교환을 n 번 한다. 비교와 교환 한 번의 시간은 각각 일정하며, 나머지 작업 시간은 무시한다.

교환 횟수 n 만 보고 전체 시간 복잡도를 $O(n)$ 이라고 해도 되는가? 이유와 올바른 빅오 표기를 쓰시오.



정답 안 된다. n^2 번 하는 비교까지 포함하면 전체 시간 복잡도는 $O(n^2)$ 이다.



해설

교환만 세면 n^2 번 하는 비교가 빠진다. 한 번의 비교와 교환에 걸리는 시간이 일정하므로 전체 시간은 n^2 에 비례하는 부분과 n 에 비례하는 부분의 합이다. n 이 커지면 n^2 부분이 주로 증가하므로 $O(n^2)$ 이다.

A는 숫자 n 개가 든 리스트이다($n=\text{len}(A)$). 다음 Python 코드의 `total + A[i] * A[j]`에서 덧셈을 실행한 총횟수를 $T(n)$ 이라 하자. 곱셈과 다른 명령은 세지 않을 때, $T(n)$ 을 n 의 식으로 쓰시오.

```
code.py

def f(A):
    n = len(A)
    total = 0
    for i in range(n):
        for j in range(n):
            total = total + A[i] * A[j]
    return total
```



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.



A는 숫자 n 개가 든 리스트이다($n=\text{len}(A)$). 다음 Python 코드의 `total + A[i] * A[j]`에서 덧셈을 실행한 총횟수를 $T(n)$ 이라 하자. 곱셈과 다른 명령은 세지 않을 때, $T(n)$ 을 n 의 식으로 쓰시오.

code.py

```
def f(A):  
    n = len(A)  
    total = 0  
    for i in range(n):  
        for j in range(n):  
            total = total + A[i] * A[j]  
    return total
```

정답 $T(n)=n^2$ 

해설

바깥 반복문 n 회마다 안쪽 덧셈이 n 회 실행되므로 총 $n \times n = n^2$ 회이다.

A는 $n \geq 1$ 개의 원소가 든 리스트이다. 다음 Python 코드에서 리스트 원소를 읽는 일과 print를 한 번 실행하는 일은 각각 일정한 시간이 든다고 가정한다. n 이 커질 때 이 함수의 시간 복잡도를 빅오로 쓰시오. 불필요하게 큰 상한은 쓰지 않는다.

```
code.py

def h(A):
    n = len(A)
    for i in range(n):
        for j in range(10):
            print(A[i], j)
```

**답**

값 · 용어 · 식으로 한 줄로 답하면 됩니다.

A는 $n \geq 1$ 개의 원소가 든 리스트이다. 다음 Python 코드에서 리스트 원소를 읽는 일과 print를 한 번 실행하는 일은 각각 일정한 시간이 든다고 가정한다. n 이 커질 때 이 함수의 시간 복잡도를 빅오로 쓰시오. 불필요하게 큰 상한은 쓰지 않는다.

code.py

```
def h(A):
    n = len(A)
    for i in range(n):
        for j in range(10):
            print(A[i], j)
```



정답 $O(n)$



해설

바깥 반복은 n 번이고, 각 반복에서 안쪽의 print는 10번 실행된다. 총 $10n$ 번이므로 시간 복잡도는 $O(n)$ 이다. 안쪽 반복 횟수 10은 n 이 커져도 변하지 않는다.

입력 n 은 2 이상의 정수이다. 다음 Python 코드의 덧셈·곱셈·비교·대입 한 번에는 각각 일정한 시간이 든다고 가정한다. n 이 커질 때 이 함수의 시간 복잡도를 빅오로 쓰시오. 불필요하게 큰 상한은 쓰지 않는다.

```
code.py

def g(n):
    count = 0
    i = 1
    while i < n:
        count = count + 1
        i = i * 2
    return count
```



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

입력 n 은 2 이상의 정수이다. 다음 Python 코드의 덧셈·곱셈·비교·대입 한 번에는 각각 일정한 시간이 든다고 가정한다. n 이 커질 때 이 함수의 시간 복잡도를 빅오로 쓰시오. 불필요하게 큰 상한은 쓰지 않는다.

code.py

```
def g(n):
    count = 0
    i = 1
    while i < n:
        count = count + 1
        i = i * 2
    return count
```



정답 $O(\log n)$



해설

i 는 1, 2, 4, 8, ...로 두 배씩 증가한다. k 번 반복한 뒤 $i=2^k$ 이며, 이 값이 n 이상이 되면 끝난다. 필요한 반복 횟수는 약 $\log_2 n$ 이므로 $O(\log n)$ 이다. 예를 들어 $n=10$ 이면 1, 2, 4, 8에서 총 4번 반복한다.

정렬할 리스트의 길이를 n 으로 표시한다. 또, 행과 열의 수가 같은 정사각행렬의 크기를 $n \times n$ 으로 표시한다.

① 리스트에서 n 은 무엇의 개수인가? ② 행렬에서 n 은 전체 칸의 수인가, 한 행의 칸 수인가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

정렬할 리스트의 길이를 n 으로 표시한다. 또, 행과 열의 수가 같은 정사각행렬의 크기를 $n \times n$ 으로 표시한다.

① 리스트에서 n 은 무엇의 개수인가? ② 행렬에서 n 은 전체 칸의 수인가, 한 행의 칸 수인가?



정답 ① 리스트 원소의 개수 ② 한 행의 칸 수(행의 수와 열의 수도 각각 n)



해설

예를 들어 3×3 행렬은 한 행에 3칸이 있고 그런 행이 3개이다. 이때 n 은 3이고 전체 칸은 9개이다. 따라서 $n \times n$ 행렬의 전체 원소 수는 n^2 개이다.

입력 크기 n 에 대한 연산 횟수가 $T(n)=n^2+n+1$ 이다. 전체 연산 횟수 중 n^2 부분이 차지하는 비율을 생각하자.
 n 이 매우 커지면 $n^2/(n^2+n+1) \times 100$ 은 몇 %에 가까워지는가?



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

입력 크기 n 에 대한 연산 횟수가 $T(n)=n^2+n+1$ 이다. 전체 연산 횟수 중 n^2 부분이 차지하는 비율을 생각하자. n 이 매우 커지면 $n^2/(n^2+n+1) \times 100$ 은 몇 %에 가까워지는가?



정답 100%



해설

분자와 분모를 n^2 으로 나누면 비율은 $1/(1+1/n+1/n^2)$ 이다. n 이 커질수록 $1/n$ 과 $1/n^2$ 이 0에 가까워지므로 이 비율은 1, 즉 100%에 가까워진다.

입력 크기 n 이 충분히 커질 때의 함수 증가 속도를 나타내는 표기이다.
 O (빅오), Ω (빅오메가), Θ (빅세타)가 각각 상한·하한·둘 모두 중 무엇을
뜻하는지 순서대로 쓰시오.



답

값 · 용어 · 식으로 한 줄로 답하면 됩니다.

입력 크기 n 이 충분히 커질 때의 함수 증가 속도를 나타내는 표기이다.
 O (빅오), Ω (빅오메가), Θ (빅세타)가 각각 상한·하한·둘 모두 중 무엇을 뜻하는지 순서대로 쓰시오.



정답

O : 상한, Ω : 하한, Θ : 상한과 하한을 모두 만족하는 같은 증가 속도



해설

비교 기준인 함수 $g(n)$ 을 생각하자. $O(g(n))$ 은 충분히 큰 n 에서 $g(n)$ 의 어떤 고정 상수배 이하라는 뜻이다. $\Omega(g(n))$ 은 어떤 양의 고정 상수배 이상이라는 뜻이다. $\Theta(g(n))$ 은 두 조건을 모두 만족하므로 $g(n)$ 과 같은 증가 속도를 나타낸다.

시간 복잡도를 쓸 때 $O(\log_2 n)$ 이나 $O(\log_{10} n)$ 을 $O(\log n)$ 으로 적어도 되는 이유를 쓰시오. 로그의 밑은 모두 1보다 큰 고정 상수이다.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

시간 복잡도를 쓸 때 $O(\log_2 n)$ 이나 $O(\log_{10} n)$ 을 $O(\log n)$ 으로 적어도 되는 이유를 쓰시오. 로그의 밑은 모두 1보다 큰 고정 상수이다.



정답 밑이 달라도 상수배 차이일 뿐이고 빅오는 상수배를 무시하기 때문이다.



해설

로그의 밑을 바꾸면 $\log_2 n = \log_{10} n / \log_{10} 2$ 이다. $\log_{10} 2$ 는 n 과 무관한 고정값이므로 두 로그는 상수배 차이이다. 빅오에서는 이런 상수배를 구별하지 않아 $O(\log n)$ 으로 쓸 수 있다.

두 알고리즘의 연산 횟수는 $T_1(n)=n^2$, $T_2(n)=n \log_2 n$ 이다. $n=1,000,000$, $\log_2 n \approx 20$ 일 때 각각의 연산 횟수를 쓰시오. 곱셈식으로 답해도 된다.



답 값 · 용어 · 식으로 한 줄로 답하면 됩니다.

두 알고리즘의 연산 횟수는 $T_1(n)=n^2$, $T_2(n)=n \log_2 n$ 이다. $n=1,000,000$, $\log_2 n \approx 20$ 일 때 각각의 연산 횟수를 쓰시오. 곱셈식으로 답해도 된다.



정답 10^{12} 회, 약 2×10^7 회



해설

$T_1=10^6 \times 10^6$, $T_2 \approx 10^6 \times 20$ 이다. 빅오만으로 실제 연산 횟수를 알 수는 없으므로 문제에 제시한 T_1 , T_2 를 이용한다.